

*Pioneers: Louis Pouzin - Notion*

# *Shell Programming with Bash*



DV1457 / DV1578 Programming in UNIX Environments

Diego Navarro, Tek.Lic.

BTH

2024

# Program

- The Shell.
- Shell Programming with Bash.
- Closing Remarks.

# *The Shell*

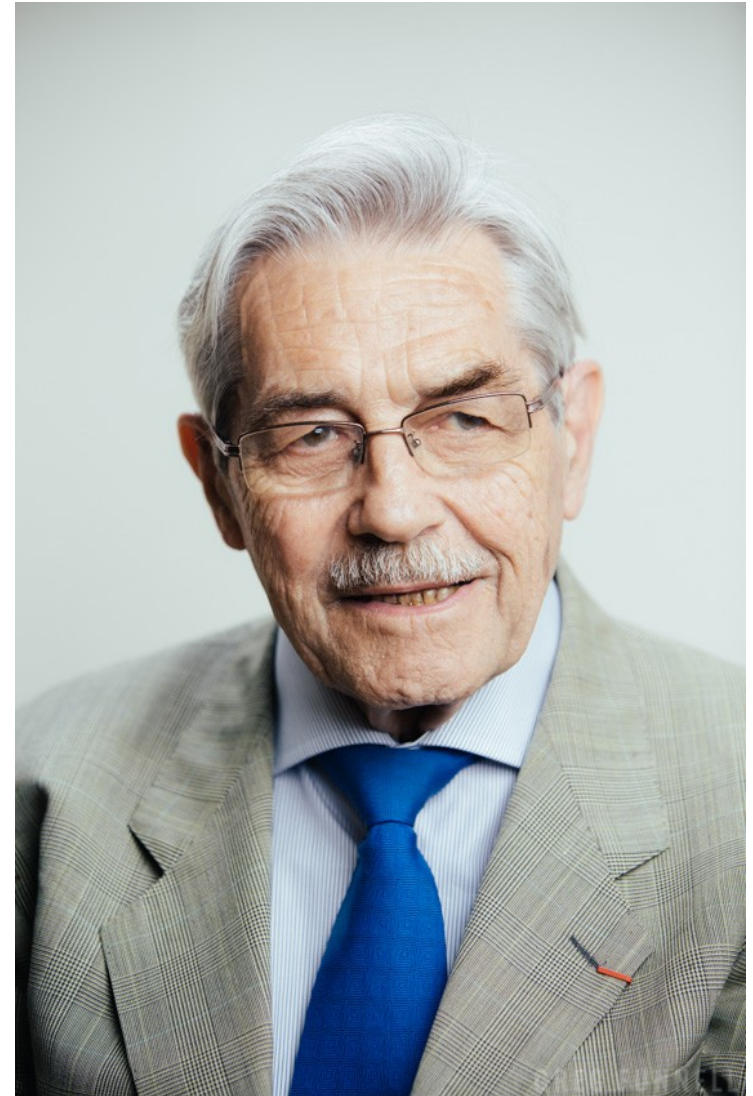


*How do we tell an operative system  
to execute specific operations?*



# The Shell

- The idea of a “Shell” was initially conceived by Louis Pouzin (MIT, 1964).
- *An interface between console messages (i.e. instructions) and subroutines (i.e. system kernel processing).*



# The Shell

- The **Shell** is a special-purpose program, designed to **read commands** written in a programming-like language, and **execute programs in response** to those commands (Kerrisk, 2010).
- It provides an interface to **interact directly with the OS**.
- The name Shell is an analogy to a sea shell.
  - Act as a layer that involves the **user-space**.

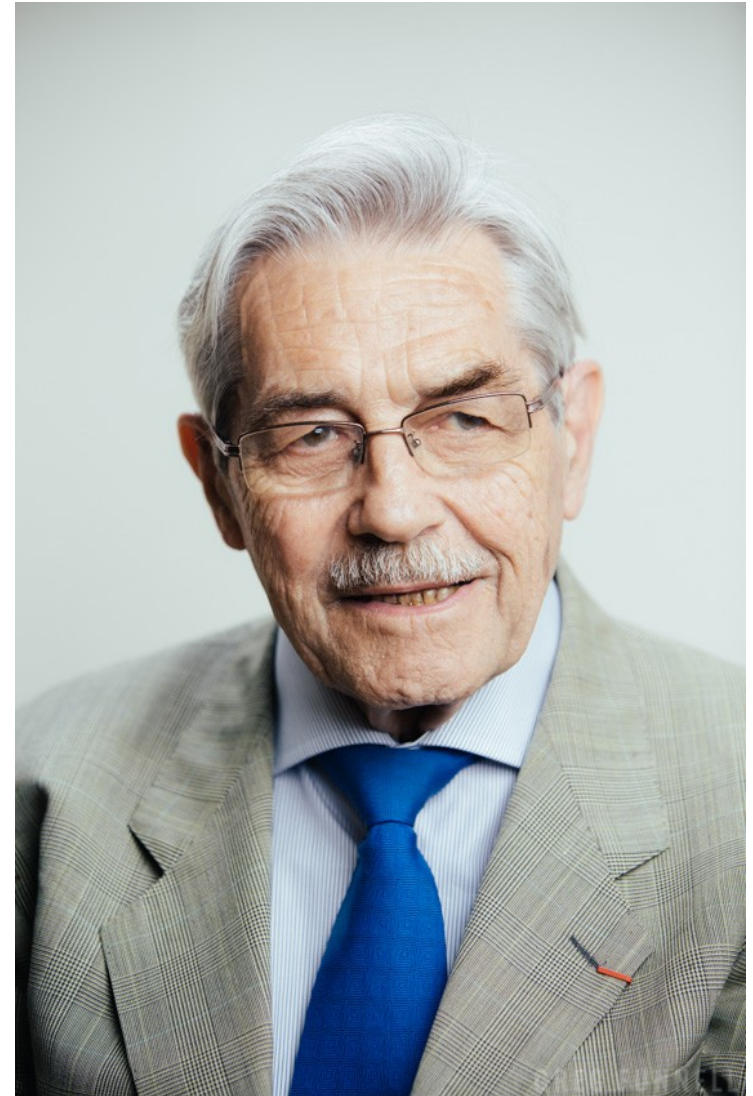
# The Shell

- Command interpretation occurs in **user-space** and emits a **syscall**.
  - Nevertheless, interpretation can also be performed in kernel-space (e.g. Plan 9 OS).
- Since Shell operates in user-space, it allows:
  - Have **multiple** shell instances in a single user session.
  - Have **multiple user shells** running **simultaneously**.



# The Shell

- The idea of a “Shell” was initially conceived by **Louis Pouzin** (MIT, 1964).
- He published a technical paper (1965), explaining how the MULTICS command language could be designed around this concept.



# The Shell

- The “First Shell”.
  - Implemented by **Glenda Schroeder** (AT&T Bell Labs, **1965**) and an unknown **General Electric** employee.
- Using Pouzin’s idea, the concept of Shell was implemented for the **MULTICS** project.



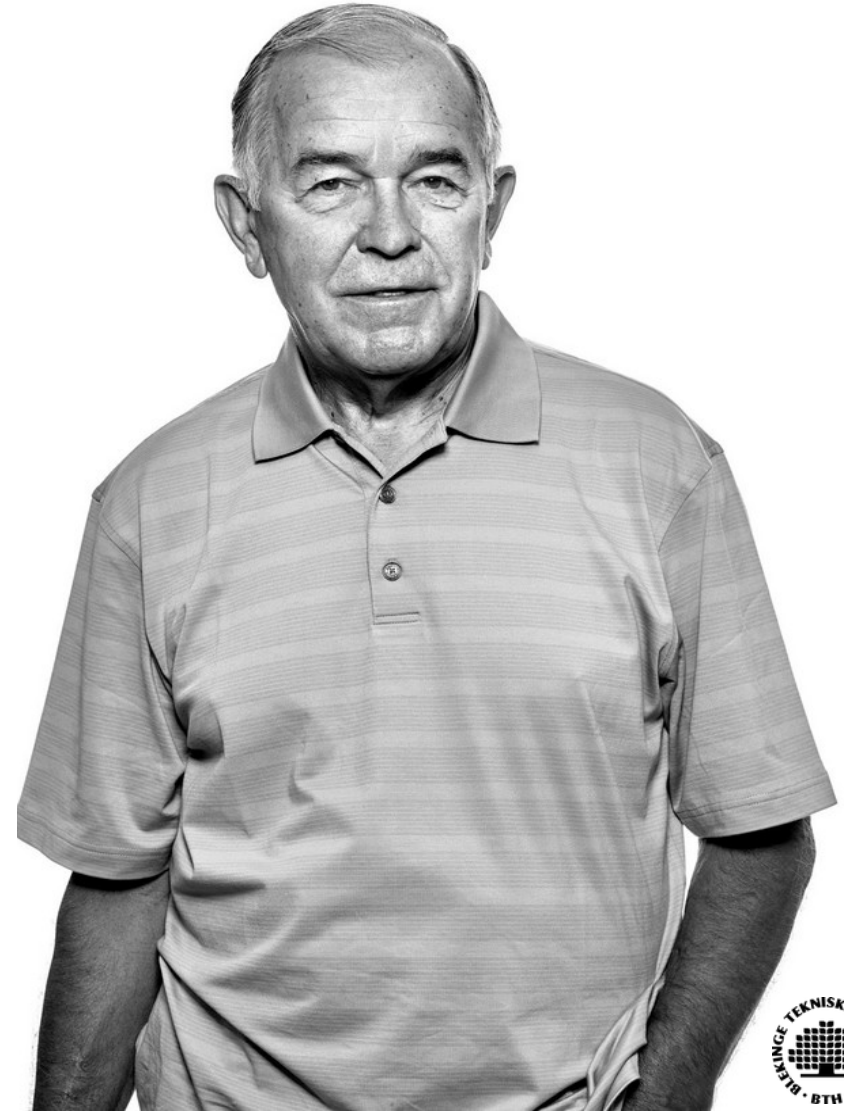
# The Shell

- The Thompson Shell.
  - Developed by Ken Thompson (AT&T Bell Labs, 1971).
  - Introduced in UNIX Version 1.
  - Modeled after the MULTICS Shell.
  - Allowed for I/O, redirection and pipes.



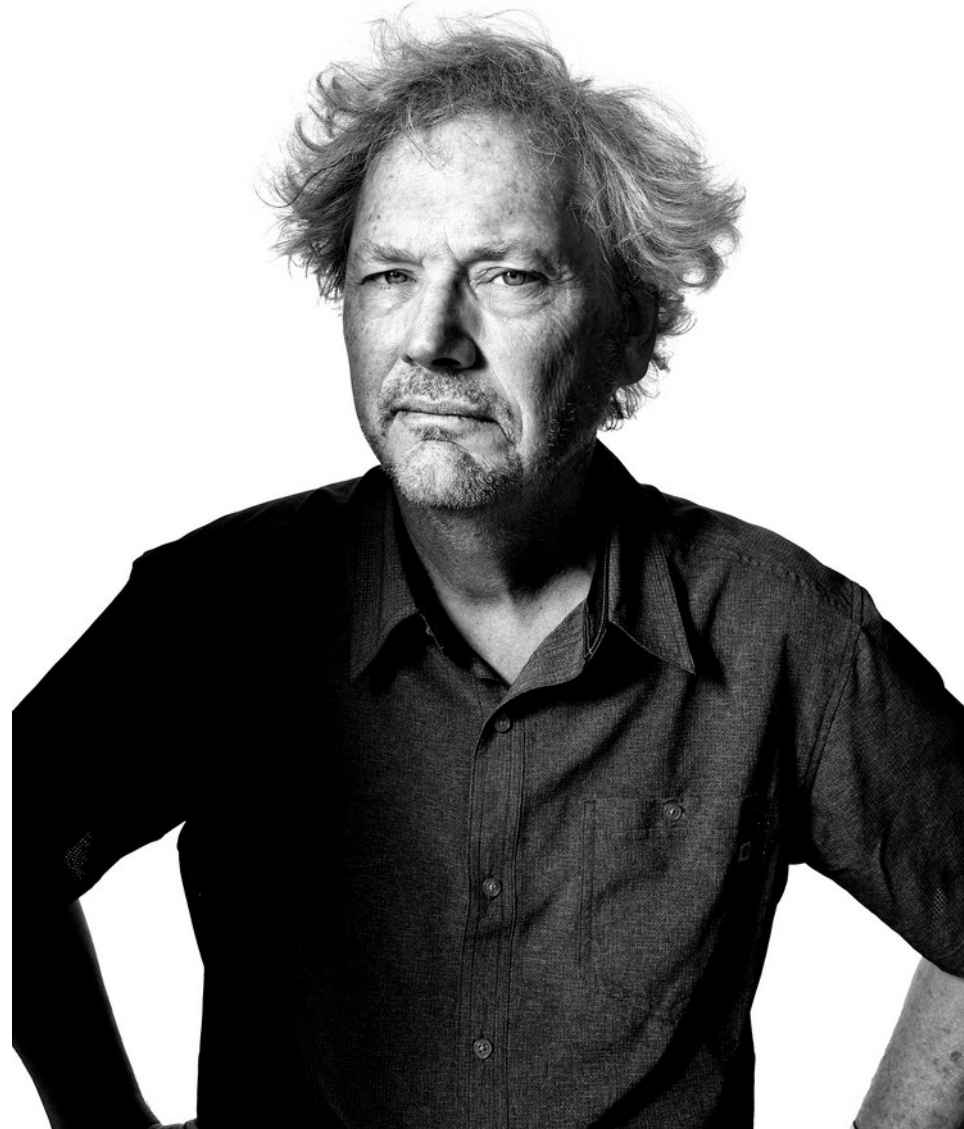
# The Shell

- The Bourne Shell (*sh*).
  - Developed by **Steve Bourne** (AT&T Bell Labs, 1979).
  - Replaced the Thompson Shell.
  - Introduced the concept of **shell scripting** (i.e. execution of files containing sets of Shell commands).
  - The standard shell for **UNIX Version 7**.
  - Commonly included in all modern implementations of UNIX.



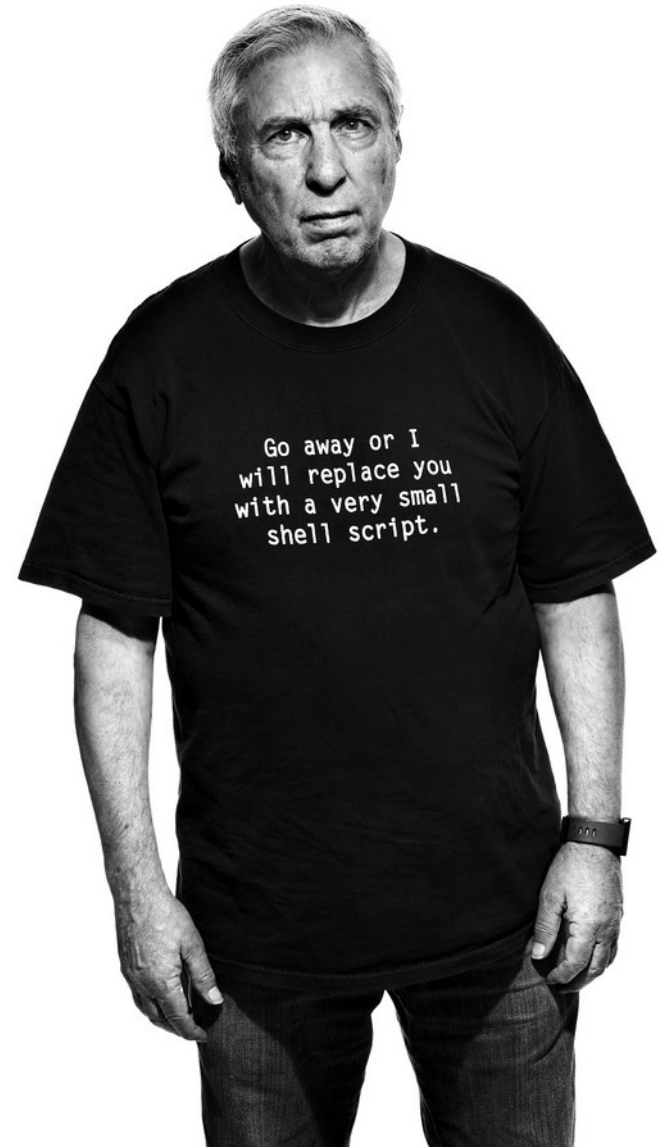
# The Shell

- The C Shell (*csh*).
  - Developed by **Bill Joy** (U. C. Berkeley, 1980).
  - Has a strong **resemblance** to the C programming language.
  - Offered **interactive features** (command history, command line editing, etc.).
  - Standard Shell for **BSD**.
  - Incompatible with the Bourne shell.
  - Did not allowed shell scripting.



# The Shell

- The Korn Shell (*ksh*).
  - Developed by **David Korn** (AT&T Bell Labs, 1983).
  - Based on (and compatible with) the Bourne Shell.
  - Added more functionality on top of *sh* (e.g. math functions, loops, inheritance and hierarchically nested variables, etc.).
  - Implemented the interactive features from the C Shell.
  - Established a Vi-style and Emacs-style syntax (i.e. popular terminal text editors).





# The Shell

- The Bourne Again Shell (*bash*).
  - Developed by Brian Fox and Chet Ramey (1989).
  - Bash is GNU Project's implementation of the Bourne Shell.
  - The most widely used Shell in GNU/Linux.



# The Shell

- The Bourne Again Shell (bash).
  - Designed to be POSIX compliant and compatible with the Bourne Shell.
  - Adopted the features from the C Shell and the Korn Shell.
  - Very robust and fully documented.



# Shell Programming with Bash



# *Basics Structures and Operations*

# Shell Programming with Bash

- Reserved words.
  - Logical and conditional: `if`, `then`, `elif`, `else`, `fi`.
  - Loops: `for`, `in`, `while`, `until`, `do`, `done`.
  - Structures: `case`, `esac`, `select`, `function`.
  - Processes: `coproc`.
  - There are also many reserved characters for arithmetic and logic operations.

# Shell Programming with Bash

- Variables.
  - Variables are set with the = operator:
    - `NAME="Diego"`
  - To indicate a reference to a variable, we use the \$ character:
    - `echo "Hello, $NAME!"` or `echo "Hello, ${NAME}!"`
    - Output: `Hello, Diego!`
  - `echo`: function to **print** information the shell.



# Shell Programming with Bash

- Variables.
  - All variables are handled as strings.
    - Non-declared variables are treated as empty strings.
    - Even numerical values are treated as string:

```
num1=10
```

```
num2=4
```

```
echo $num1+$num2
```

- Output: 10+4

# Shell Programming with Bash

- Variables.
  - `expr`: command for evaluating integer expressions.
  - Arithmetic operations:

```
expr 5 + 3
expr 7 - 2
expr 12 / 3
expr 4 \* 6
```
- Multiplication (\*) requires the use of a **backslash** (\).
  - \ is used to **escape special characters**. If we don't use it here, bash will think we are using a wildcard instead of a multiplication.

# Shell Programming with Bash

- Variables.
- Rewriting our code we get:

```
num1=10
```

```
num2=4
```

```
expr $num1 + $num2
```

- Output: 14
- `bc`: basic calculator → `echo "3.14 * 2.0" | bc`

# Shell Programming with Bash

- Logical and Conditional Operations.
  - **if**, **then**: declares a conditional statement and its respective instruction.
  - **else**: specifies the instruction that executes after a preceding if statement is false.
  - **elif**: (else if) allows to check multiple conditional statements sequentially.
  - **fi**: finishes an if statement block.

# Shell Programming with Bash

- Logical and Conditional Operations.
  - **Integers** relational operators:
    - Equal to: **-eq**
    - Not equal to: **-ne**
    - Greater than: **-gt**
    - Greater than or equal to: **-ge**
    - Less than: **-lt**
    - Less or equal to: **-le**
  - Squared brackets [ ] are used to denote the **test** command.

# Shell Programming with Bash

- Logical and Conditional Operations.

```
age=25
if [ $age -ge 18 ]; then
    echo "You are an adult."
else
    echo "You are not yet an adult."
fi
```

- The ; works as an statement separator.

# Shell Programming with Bash

- Logical and Conditional Operations.

```
grade=75
if [ $grade -ge 90 ]; then
    echo "You got an A!, Amazing!"
elif [ $grade -ge 80 ]; then
    echo "You got a B, Baffling."
elif [ $grade -ge 70 ]; then
    echo "You got a C, Concerning."
else
    echo "I'm not mad, just Disappointed."
fi
```



# Shell Programming with Bash

- Logical and Conditional Operations.
  - **String** relational operators:
    - Equal to: =
    - Not equal to: !=
    - Greater than (alphabetically): >
    - Lesser than (alphabetically): <

# Shell Programming with Bash

- Logical and Conditional Operations.

```
user="awesome_sysadmin"  
password="admin123"
```

```
if [ "$user" = "awesome_sysadmin" ]; then  
    if [[ "$password" == "admin123" ]]; then  
        echo "Access granted."  
    else  
        echo "The password is not strong enough."  
    fi  
else  
    echo "The user is not awesome."  
fi
```

# Shell Programming with Bash

- Logical and Conditional Operations.

```
string1="hello"  
string2="world"
```

```
if [[ "$string1" != "hello" ]]; then  
    echo "String 1 is not equal to 'hello'"  
elif [[ "$string2" != "world" ]]; then  
    echo "String 2 is not equal to 'world'"  
else  
    echo "The string matches 'hello world'."  
fi
```

# Shell Programming with Bash

- Logical and Conditional Operations.

```
word1="Argentina"  
word2="France"
```

```
if [[ "$word1" > "$word2" ]]; then  
    echo "Alphabetically '$word2' comes first, and '$word1' comes second"  
else  
    echo "Alphabetically '$word1' comes first, and '$word2' comes second."  
fi
```

- When comparing strings, the value of letters increases according to their position in the alphabet.

# Shell Programming with Bash

- Logical and Conditional Operations.
  - File relational operators:
    - Exist: `-e`
    - Directory: `-d`
    - Read permissions: `-r`
    - Write permissions: `-w`
    - Execute permissions: `-x`

# Shell Programming with Bash

- Logical and Conditional Operations.

```
if [ -e output.txt ]; then
    echo "The file exists."
else
    echo "The file doesn't exist."
fi
```

# Shell Programming with Bash

- Logical and Conditional Operations.

```
if [ -x output.txt ]; then
    echo "The file can run as an application."
else
    echo "The file cannot run as an application."
fi
```

# Shell Programming with Bash

- Loop Operations.
  - `for`, `in`: declares a for-loop and establishes the instruction for repetition.
  - `while`: declares a while-loop and the logical condition to continue it.
  - `until`: declares the until-loop and the logical condition to interrupt it.
  - `do`, `done`: determines the start and the end of the loop-action.



# Shell Programming with Bash

- Loop Operations.

```
for i in {1..5}; do
    echo "Number: $i"
done
```

- Brace expansion `{..}`: creates a sequence with values within the curved braces.
  - `{1..5}` → Values from 1 to 5.
  - `{A..Z}` → Values from A to Z.
  - `{1..10..2}` → Values from 1 to 10, with increments of 2.

# Shell Programming with Bash

- Loop Operations.

```
count=0
while [ $count -lt 5 ]; do
    echo "Count: $count"
    ((count++))
done
```

- Arithmetic operator (( )): determines a numerical expression in Bash.
  - ((result = 5+3))

# Shell Programming with Bash

- Loop Operations.

```
count=0
until [ $count -eq 5 ]; do
    echo "Count: $count"
    ((count++))
done
```

# Shell Programming with Bash

- Structures.
  - **case**, **esac**: declares the beginning and the end of a case statement.
  - **select**: declares a set of options for the user to select.
  - **function**: declares a reusable section of code.
    - **{ }**: defines the body (code) of the function.

# Shell Programming with Bash

- Structures.

```
select choice in "Option 1" "Option 2" "Exit"; do
    case $choice in
        "Option 1") echo "You selected Option 1." ;;
        "Option 2") echo "You selected Option 2." ;;
        "Exit") break;;
        *) echo "Invalid choice." ;;
    esac
done
```

# Shell Programming with Bash

- Structures.

```
function my_name_is() {  
    echo "Hello, my name is $1, and I am $2 years old."  
}
```

```
my_name_is "Diego" 38
```

# Shell Programming with Bash

- Shell Scripting.
  - Execution of a text file that contains a sequence of Bash commands.
  - A shell scrip must always start with a **shebang** (**#!**): line that **specifies** which **interpreter** must be used to execute the file.
    - The shebang is followed by the path to the desired interpreter (**/bin/bash**) in the case **for bash**.
  - A bash script uses the **.sh** file extension.

# Shell Programming with Bash

- Shell Scripting.

- File name: `my_name.sh`

- Content:

```
#!/bin/sh
```

```
function my_name_is() {  
    echo "Hello, my name is $1, and I am $2 years old."  
}
```

```
my_name_is "Diego" 38
```



# Shell Programming with Bash

- Shell Scripting.
  - There are two ways to execute a shell script.
  - The **bash command**: we indicate the shell that should execute the file as a series of bash instructions:

```
bash my_name.sh
```

# Shell Programming with Bash

- Shell Scripting.
  - Granting **execution permission**: We tell the OS that the file `my_name.sh` is allowed to be **executed like** any other **application** with:  
`chmod +x my_name.sh`
  - After granting execution permissions, we can execute the file with the following command:  
`./my_name.sh`

# Shell Programming with Bash

- Shell Scripting.
  - `chmod` (change mode): command to change file permissions.
  - Permissions in the GNU/Linux OS:
    - `-r` for read.
    - `-w` for write.
    - `-x` for execute.
  - Keep scripts as simple as possible.
  - A single task per script!

# *Additional Functionality*

# Shell Programming with Bash

- Environmental Variables.
- Let's run the following script named `dm.sh`:

```
#!/bin/bash  
message="This is a message"  
echo $message
```

- Output: `This is a message`

# Shell Programming with Bash

- Environmental Variables.
  - However, if after running the script I run in the shell `echo $message`, it returns nothing.

*Why does this happen?*

# Shell Programming with Bash

- Environmental Variables.
  - Shell will print an empty string, since it doesn't know what the `$message` variable is (it only exists in the context of the script, but not in the context of the shell)
  - By default, the scope of the variable is the scrip that contains it.



# Shell Programming with Bash

- Environmental Variables.
  - We can declare environment variables (i.e. variables that are accessible by child processes or other shell sessions) using the export command:

```
export message="This is a message"
```

- Having done this, **\$message** will be available to scripts executed in the current shell session.
  - Therefore, we don't need to declare it in the script!

# Shell Programming with Bash

- Environmental Variables.
  - We can now update `dm.sh` to:  
`echo $message`
  - We can also do all this in a single line:  
`export message="This is a message"; bash dm.sh`

# Shell Programming with Bash

- Environmental Variables.
  - We can also **load** variables, functions, and other configurations from a script into the current shell session using the **source** command.
    - **source** is commonly used with the `.` character (i.e. a dot).
  - Let's reuse the example using the first version of `dm.sh`:

```
#!/bin/bash
message="This is a message"
echo $message
```

# Shell Programming with Bash

- Environmental Variables.
  - We can turn the `$message` variable in the script into a environment variable using source:  
`source dm.sh` or `. dm.sh`
  - Once this is done, we can simply print the variable directly from the shell:  
`echo $message`  
Output: `This is a message`

# Shell Programming with Bash

- Wildcards.
  - Special **characters** used to **match patterns** in file or directory names:
    - **?** : replaces a single unknown character.
    - **\*** : replaces several unknown characters.
    - **[ ]** : matches any specific character or range of characters.

# Shell Programming with Bash

- Wildcards.

```
ls report_2024_?.xls
```

- Output: all file in the current directory that matches the prefix **report\_2024\_** with only one character (e.g. **report\_2024\_1.xls**, **report\_2024\_5.xls**, etc.).
- **ls**: command to display the contents of a directory.

# Shell Programming with Bash

- Wildcards.

```
ls *.txt
```

- Output: a list of all .txt files in the current directory.

```
ls report*.pdf
```

- Output: all .pdf files in the current directory, that start with the prefix “report” (e.g. **report**01.pdf, **report**\_02.pdf, **reporting**\_march.pdf, etc.).

# Shell Programming with Bash

- Wildcards.

```
ls [Report]*.*
```

- Output: a list of all files that start with the word Report (**Report**\_01.xls, **Report**2.txt, **Report**ing\_grades.pdf).
- And yes, we can combine wildcards together!



# Shell Programming with Bash

- I/O Redirection.
  - Allows to **control the flow of data** between your **shell commands** and:
    - The **stdin** (i.e. standard input or the data source (keyboard)).
    - The **stdout** (i.e. standard output or the result destination (terminal)).
    - The **stderr** (i.e. standard error messages and non-standard outputs).

# Shell Programming with Bash

- I/O Redirection.
  - `<` (`stdin`) : redirects data input to another source.

```
while read -r line; do
echo "Line: $line"
done < words.txt
```

- Output: prints, line by line, the contents in the file `words.txt`.

# Shell Programming with Bash

- I/O Redirection.
  - `>` (`stdout`) : redirects data output to a different destination.

```
echo "Hello world" > output.txt
```

- Output: stores the line `"Hello world"` into the file `output.txt` (it creates the file if it doesn't exist).

```
echo "Hello again" > output.txt
```

- Output: if the file `output.txt` already exists, this command overwrites it content and replaces it with `"Hello again"`.

# Shell Programming with Bash

- I/O Redirection.

- `>> (stdout)` : appends data output to a different destination.

```
echo "Hello world" > output.txt
```

- Output: stores the line `"Hello world"` into the file `output.txt` (it creates the file if it doesn't exist).

```
echo "Hello again" >> output.txt
```

- Output: if the file `output.txt` already exists, this command adds `"Hello again"` as a new line in the file.

# Shell Programming with Bash

- I/O Redirection.
  - `2> (stderr)` : redirect error messages to a different destination.

```
ls output.txt
```

- Output: `output.txt` (it list a file with that name)

```
ls nonexistent_file.plop
```

- Output: `ls: cannot access 'nonexistent_file.plop': No such file or directory`

# Shell Programming with Bash

- I/O Redirection.

```
ls nonexistent_file.plop 2> error.txt
```

- Output: the file **error.txt** is created, containing the error message.

- We can also combine **stdout** and **stderr** if needed using **2>&1**:

```
echo "Hello world" > output.txt 2>&1
```

- Output: the file **output.txt** will contain the “Hello World” line, together with any execution error message.

# Shell Programming with Bash

- Pipes.
  - Pipes ( | ) are conduits between commands (i.e. they use the output of one command as the input for another).
  - `ls -l | grep "txt"`
  - Output: from the files in the current directory, all the files containing **txt** will be listed.
- **grep** searches plain-text data sets that match a specific pattern.

# Shell Programming with Bash

- Internal Field Separators.
  - Special variables that determine how bash splits strings into words (i.e. string delimiters). It uses 3 variables by default:
    - Space ( ' ' )
    - Tab ( '\t' )
    - Newline ( '\n' )
  - We can set a custom variable to be used a delimiter.



# Shell Programming with Bash

- Internal Field Separators.

```
Read a b c (user inputs: Banana Apple Cherry)
```

```
echo "a=$a"
```

```
echo "b=$b"
```

```
echo "c=$c"
```

- Output: `a=Banana b=Apple c=Cherry`

- **Read:** Reads inputs from users or files and stores it in variables.

# Shell Programming with Bash

- Internal Field Separators.

```
Read a b c (user inputs: Banana,Apple,Cherry)
```

```
echo "a=$a"
```

```
echo "b=$b"
```

```
echo "c=$c"
```

- Output: a=Banana,Apple,Cherry (other variables will be empty)

# Shell Programming with Bash

- Internal Field Separators.

```
IFS=" , "
```

```
Read a b c (user inputs: Banana,Apple,Cherry)
```

```
echo "a=$a"
```

```
echo "b=$b"
```

```
echo "c=$c"
```

- Output: a=Banana b=Apple c=Cherry

# *Useful External Programs*

# Useful External Programs

- **grep**: search plain-text for lines that match a regular expression.
- **sed**: a stream-editor for manipulating and transforming text.
- **awk**: text processing toolkit
- **cut**: extract specific lines from text.
- **wc**: counts words, lines, or characters.
- **find**: search for files and directories within the file hierarchy.
- **cat**: concatenates (i.e. displays contents of files in the terminal).
- **man**: Manual. It access the documentation of commands and utilities

# *Closing Remarks*



# Closing Remarks

- There is so much more to bash!
  - When in doubt, use the **man** command!
- The Shell is one of the most representative tools from UNIX/Linux.
  - When used correctly, there is no faster way to modify and operate with the OS.
- Shell programming is a particularly powerful tool for automation and system administration.

# Closing Remarks

- Key idea in bash scripting: **Modularity**.
  - Specific scripts for specific tasks.
  - Interconnect scripts and data with pipes and redirection.
  - Stream text as input and output.



# Closing Remarks

- For this course we will be using [Fedora 40 KDE Spin](#).
- We recommend a Type-2 hypervisor (e.g. [Oracle's Virtual Box](#)) to install and run this OS.
  - Feel free to use any hypervisor you are familiar or comfortable with.

# Homework

- Familiarize yourself with the examples shared in this lecture.
  - ⚠ Pay close attention to syntax and character spacing.
  - Test them in Fedora 40 using the terminal emulator “**Konsole**”.
  - Install a text editor called “**Kate**” using “Discover”, Fedora’s software center.
- We will expand over these concepts in our next Lab.



**WHAT?**